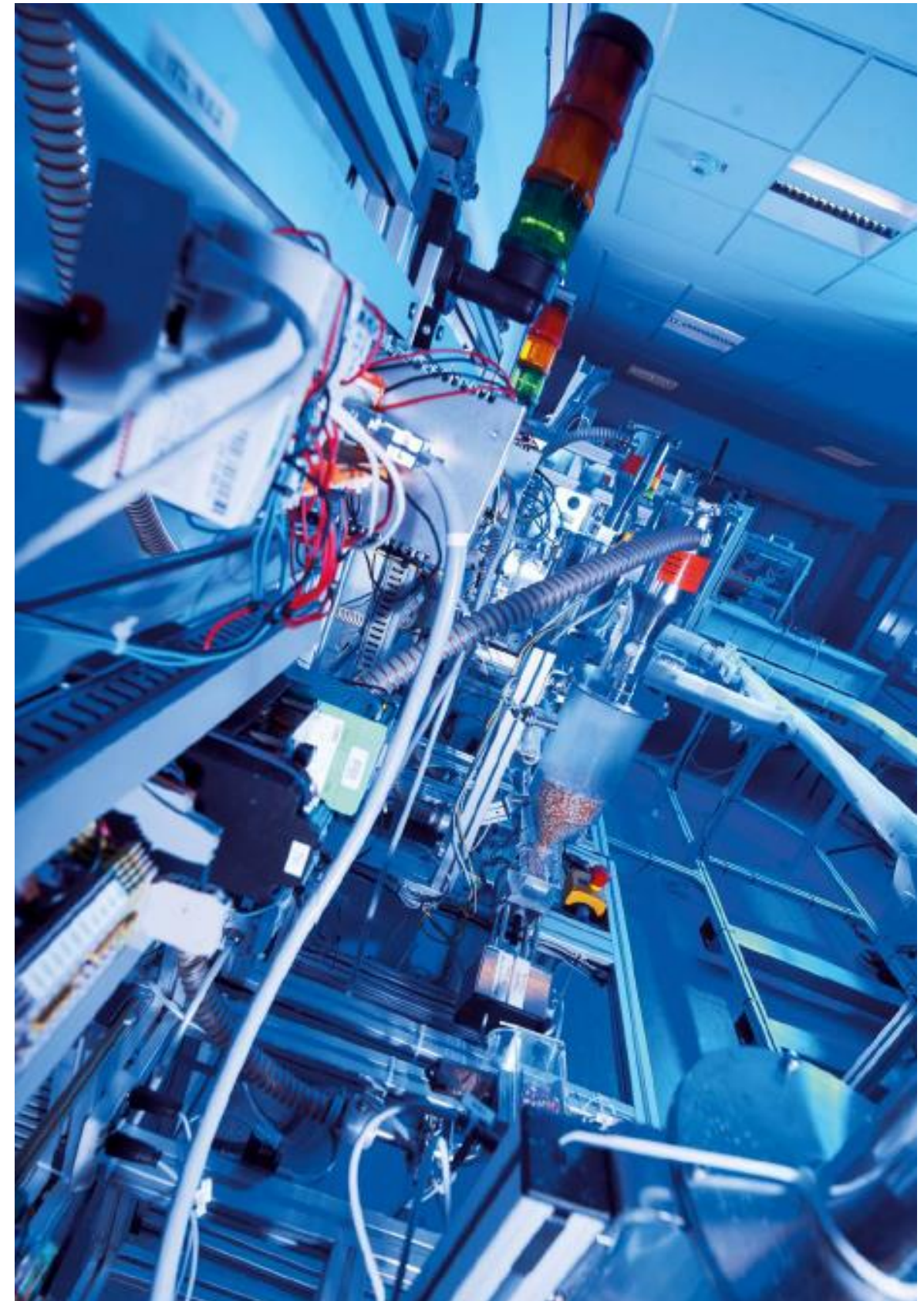


Echtzeit-Datenverarbeitung

Peripherie

Prof. Dr. Henning Trsek

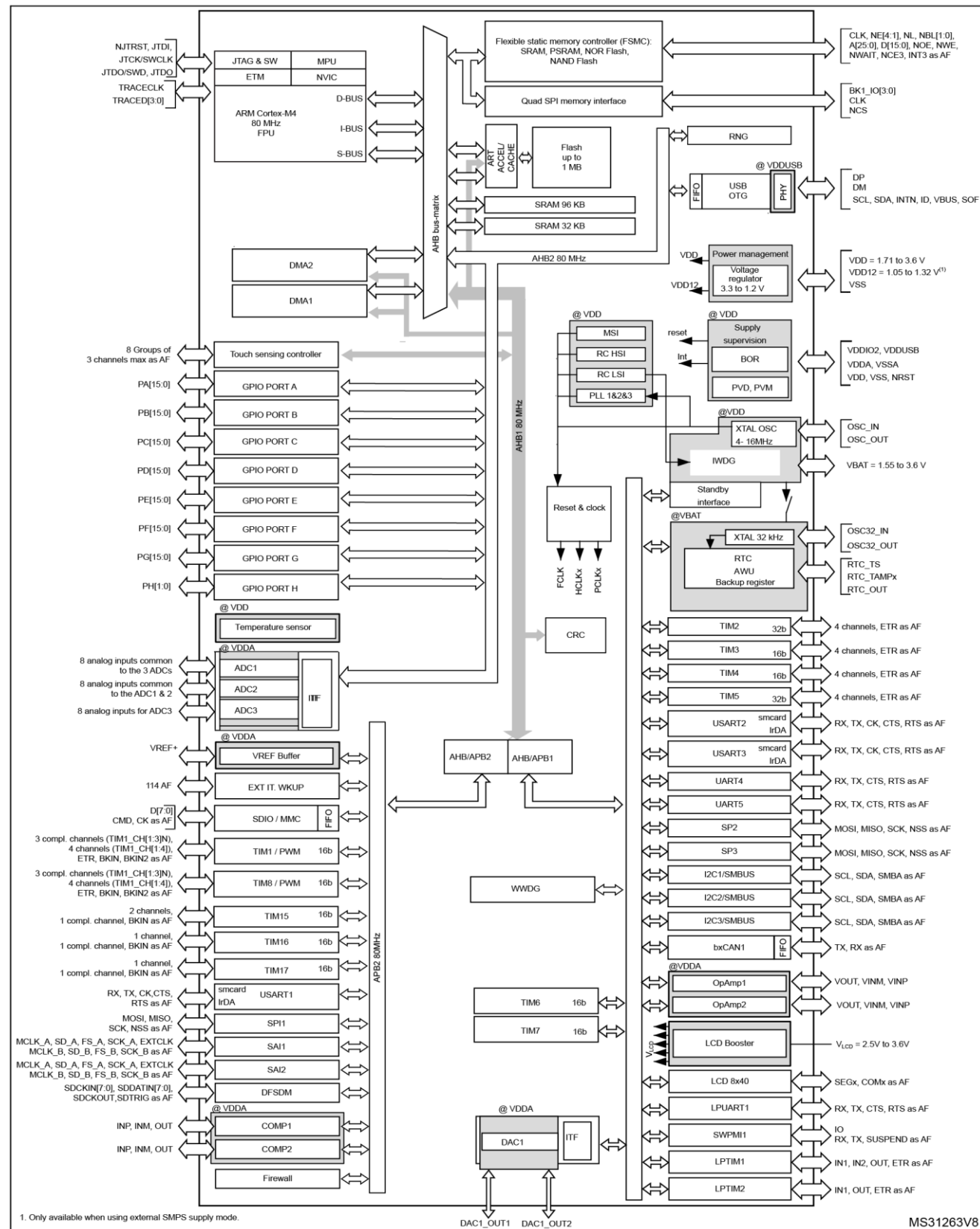
Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



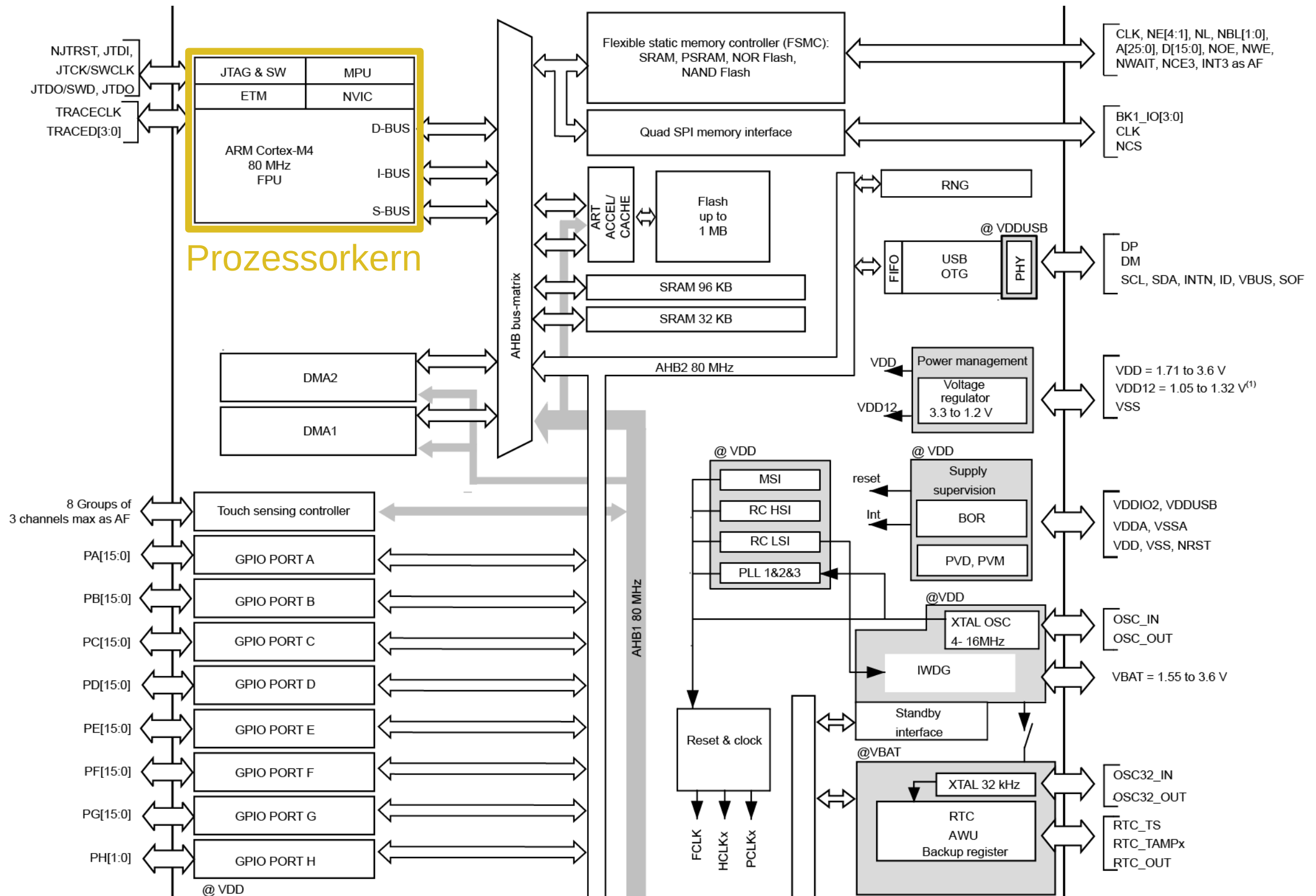
Überblick Peripherie

- ▶ C Programmierung (Wiederholung)
 - Datentypen
 - Operatoren
 - Zeiger
- ▶ Digitale Ein-/Ausgabe
- ▶ Timer und PWM
- ▶ Funktionen

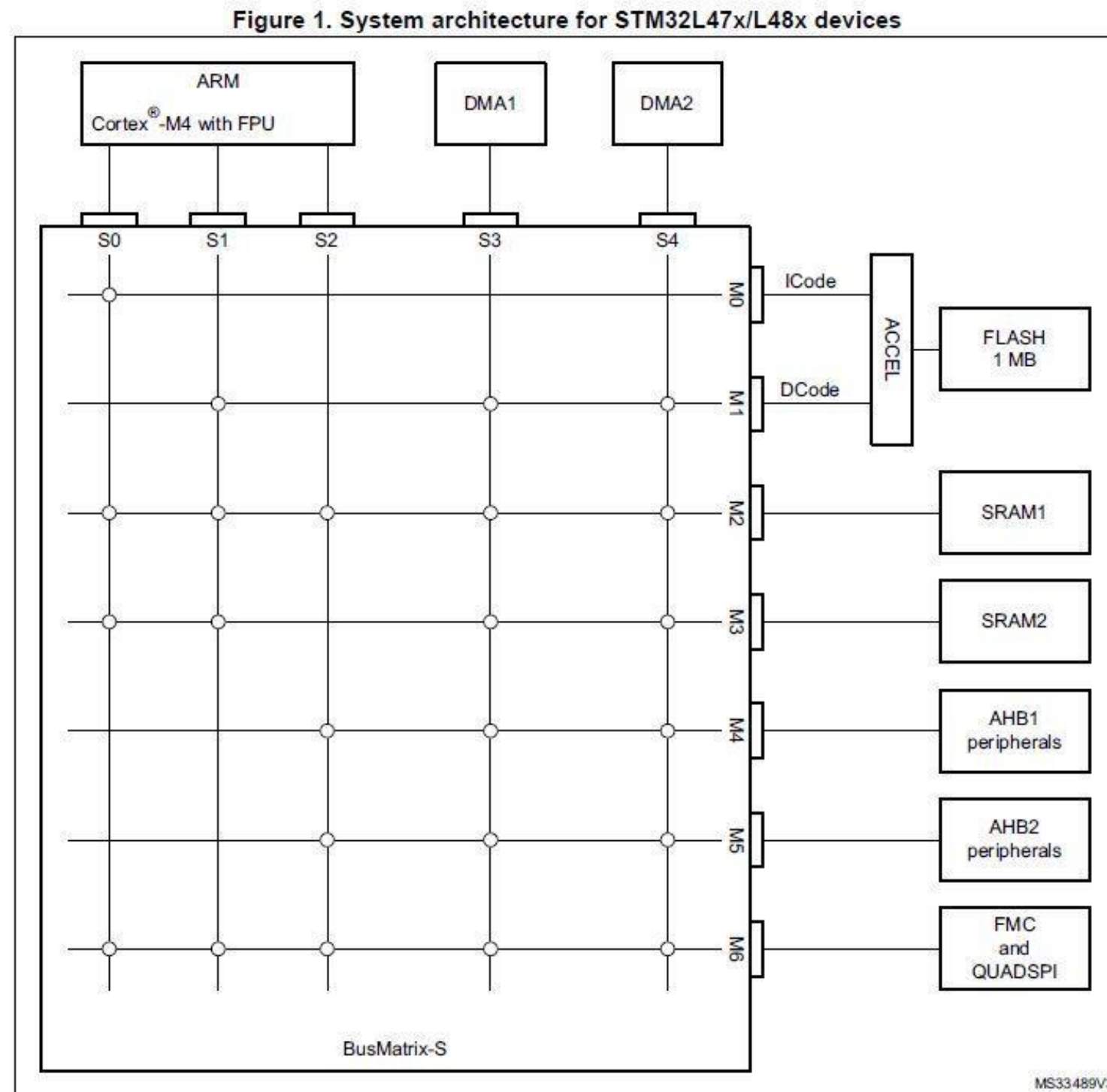
Zugriff auf Digital-Ein-/Ausgabe



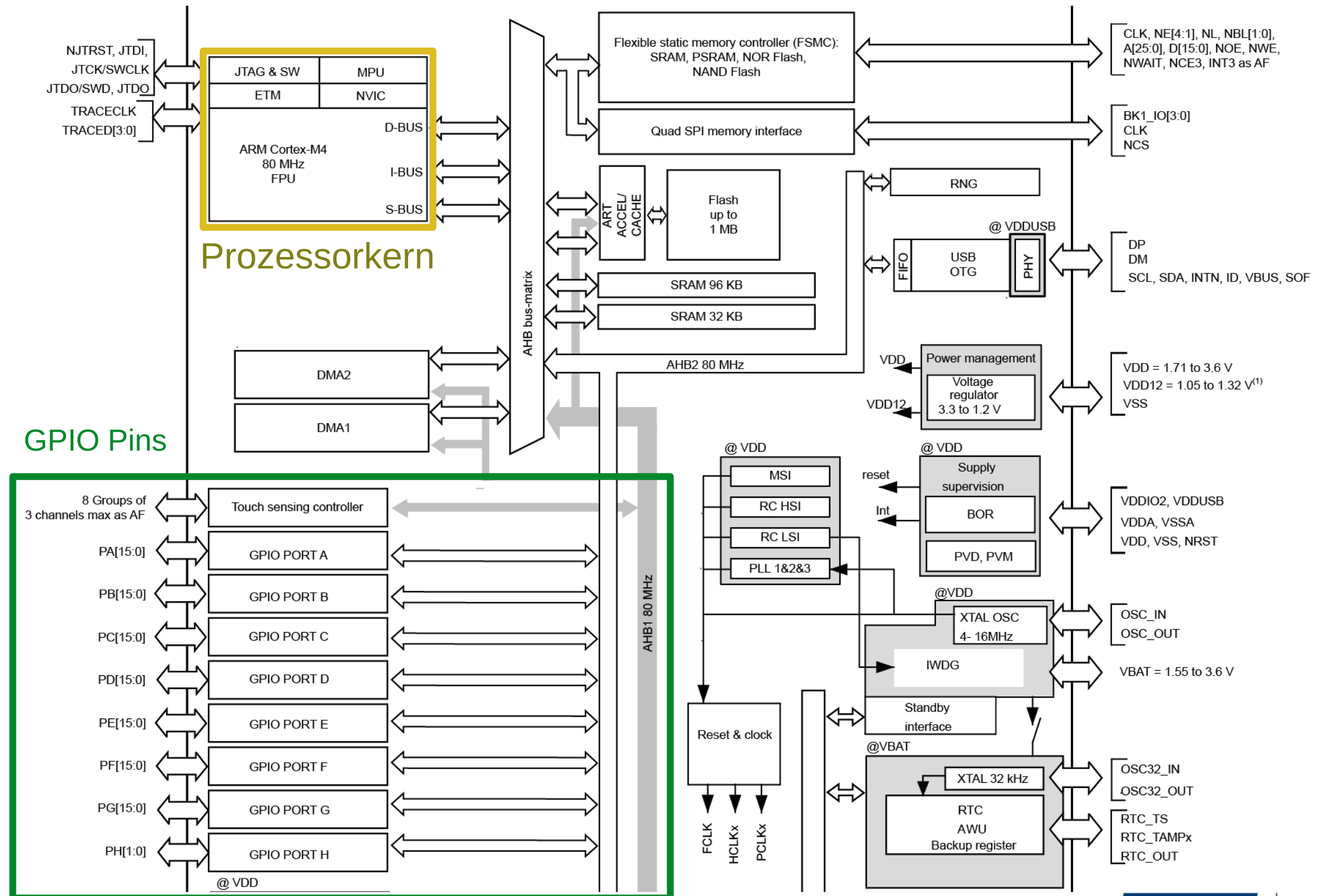
Zugriff auf Digital-Ein-/Ausgabe



Zugriff auf Digital-Ein-/Ausgabe



Zugriff auf Digital-Ein-/Ausgabe

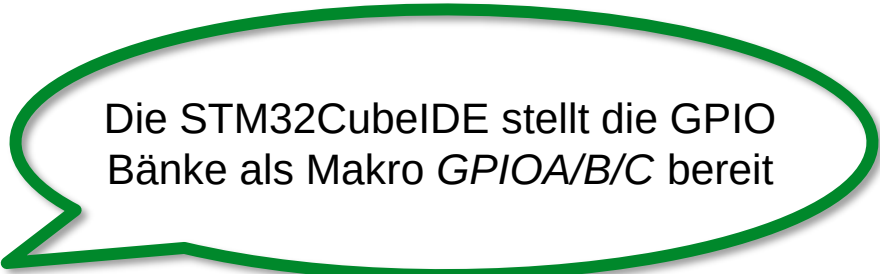


Zugriff auf Digital-Ein-/Ausgabe

Zugriff auf GPIO-Pins:

Direkter Speicherzugriff über eigene Struktur / Makros:

```
long eingang = GPIOA->IDR; //Lesen
GPIOB->ODR   = eingang;    //Schreiben
```



Die STM32CubeIDE stellt die GPIO Bänke als Makro *GPIOA/B/C* bereit

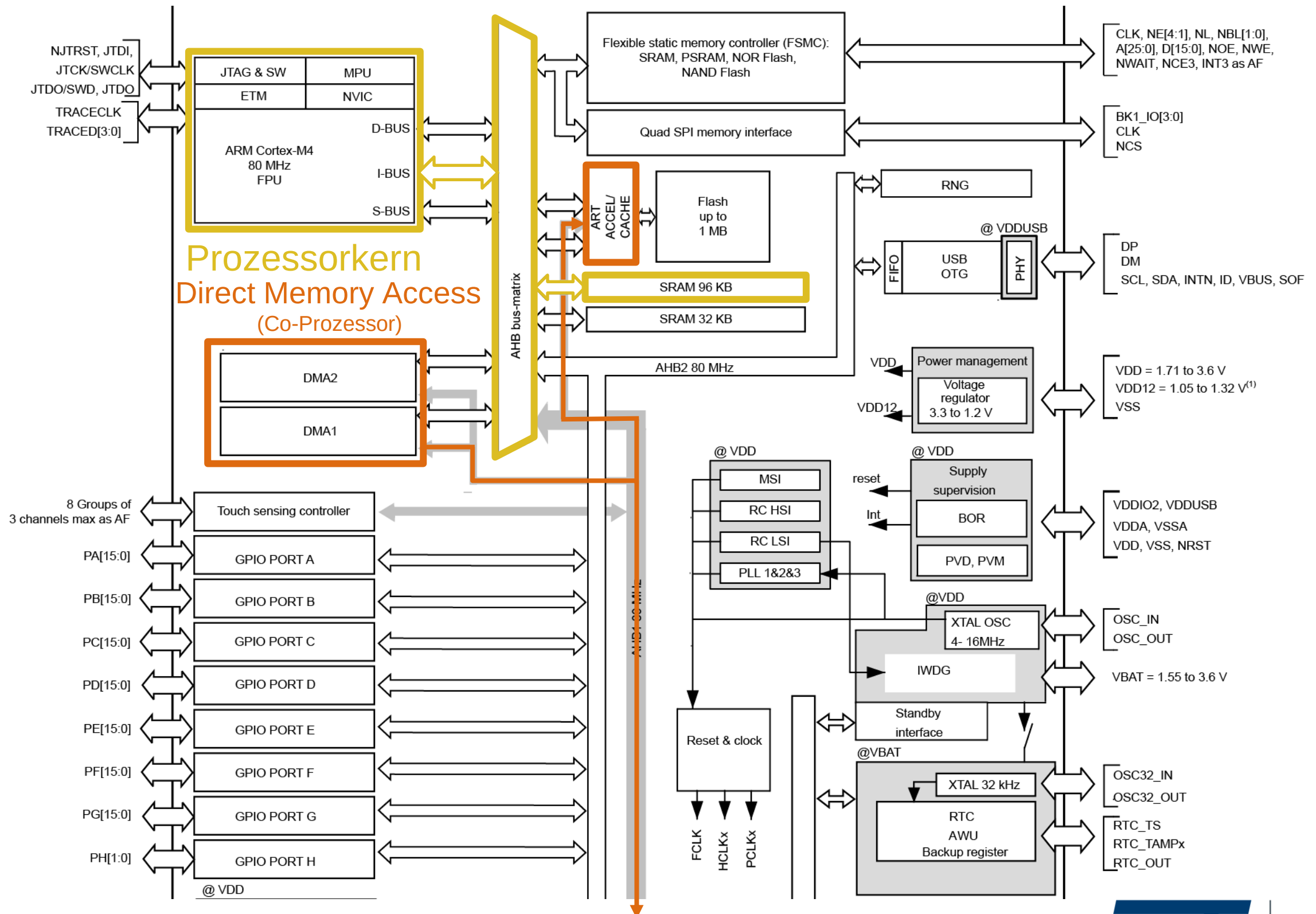
- Ganze GPIO Bank wird gleichzeitig gelesen
- Einzelne Pins müssen über Masken gefiltert werden
- Der Programmierer ist dafür verantwortlich nur die beabsichtigten Pins zu schreiben

Nutzung der Hardware Abstraction Layer (HAL) Bibliothek:

```
HAL_GPIO_ReadPin(GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin);
HAL_GPIO_WritePin(GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState);
HAL_GPIO_TogglePin(GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin);
```

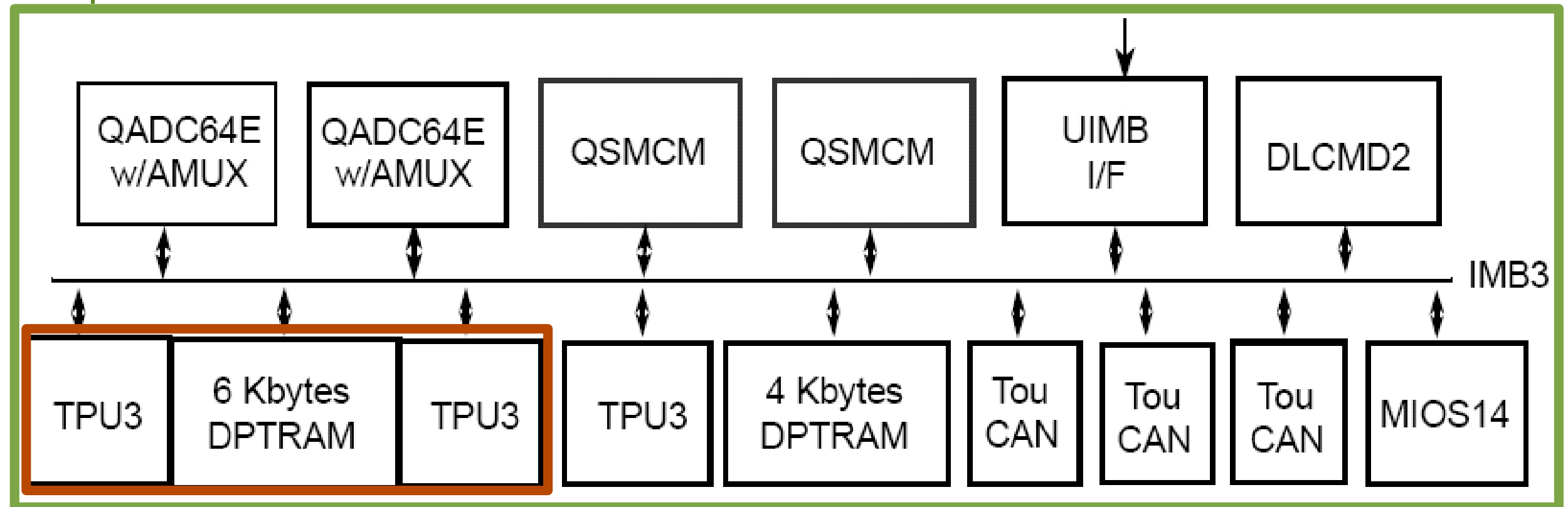
- Einzelne Pins können direkt angesprochen werden
- Schutz vor unbeabsichtigten Änderungen
- Für jeden Pin ist ein separater Aufruf nötig

Zugriff auf Digital-Ein-/Ausgabe



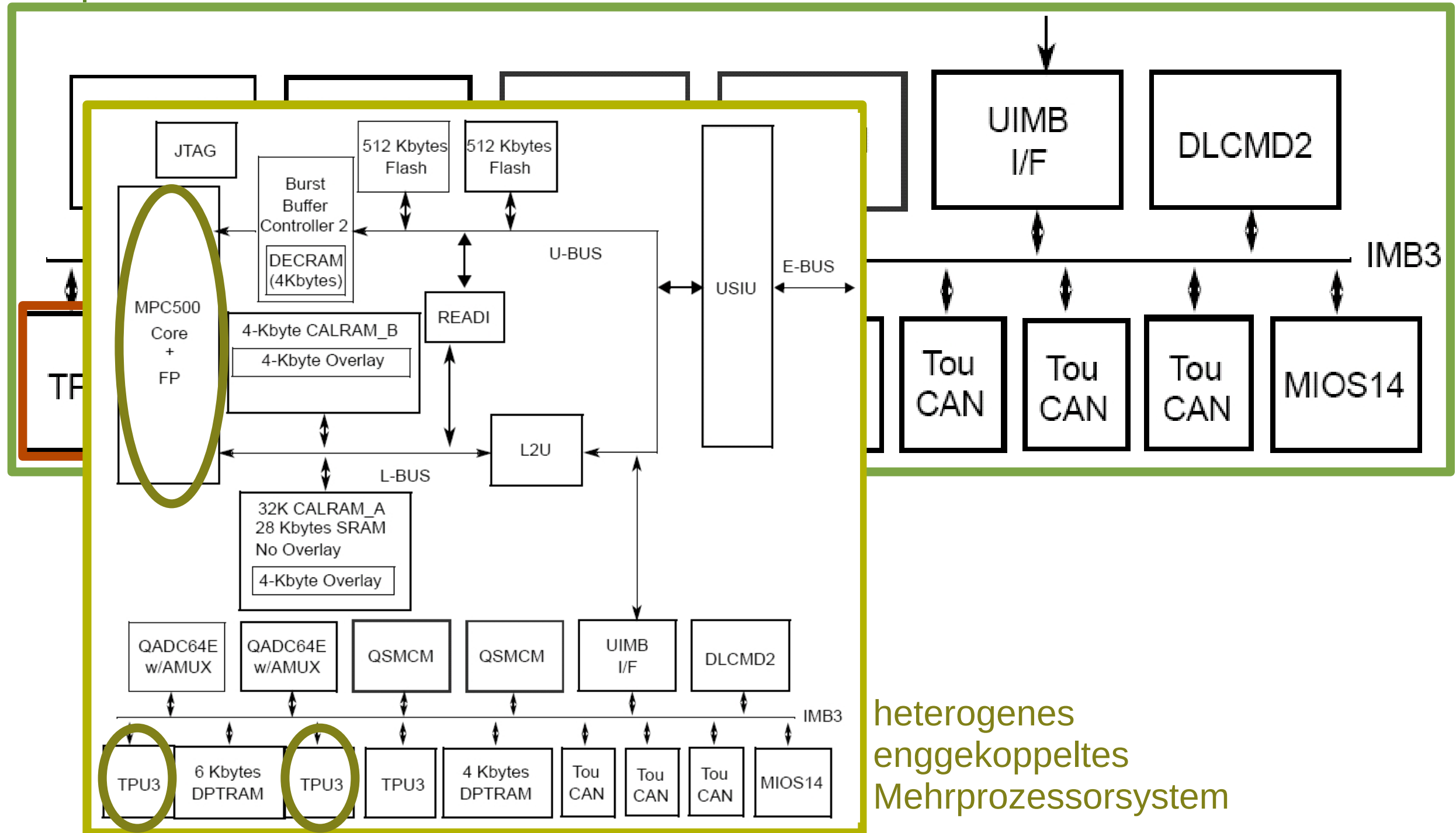
Zugriff auf Digital-Ein-/Ausgabe mit Co-Proz.

Peripherie



Zugriff auf Digital-Ein-/Ausgabe

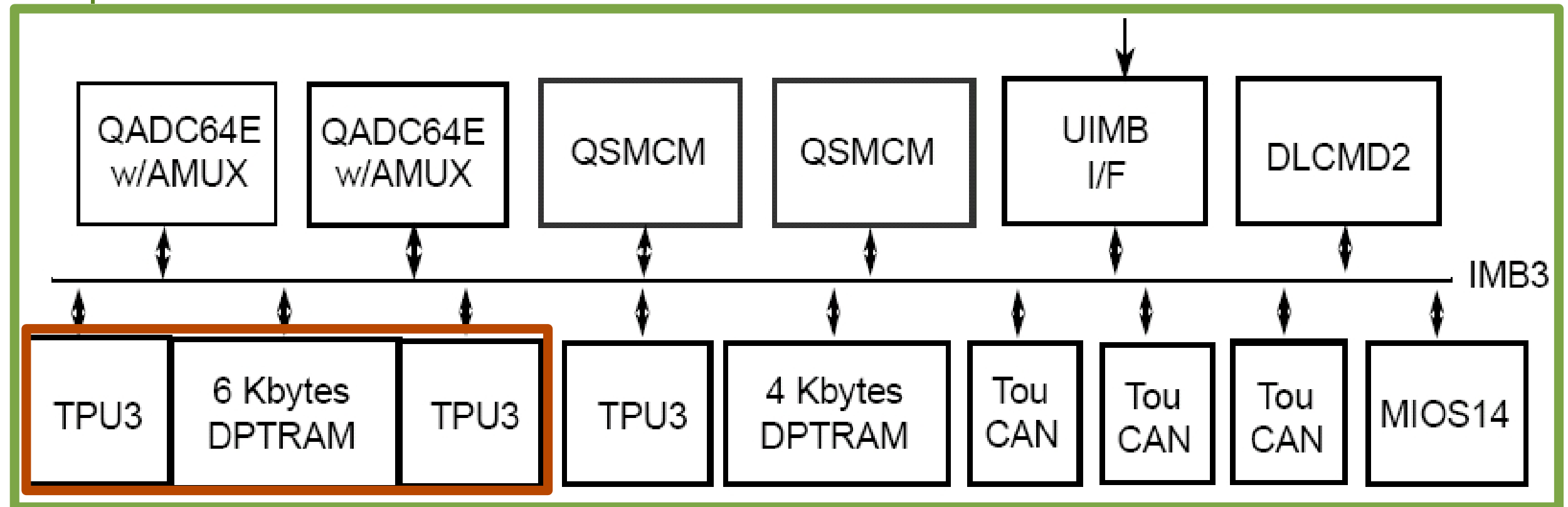
Peripherie



heterogenes
enggekoppeltes
Mehrprozessorsystem

Zugriff auf Digital-Ein-/Ausgabe

Peripherie



Time Processor Unit

Digital I/O

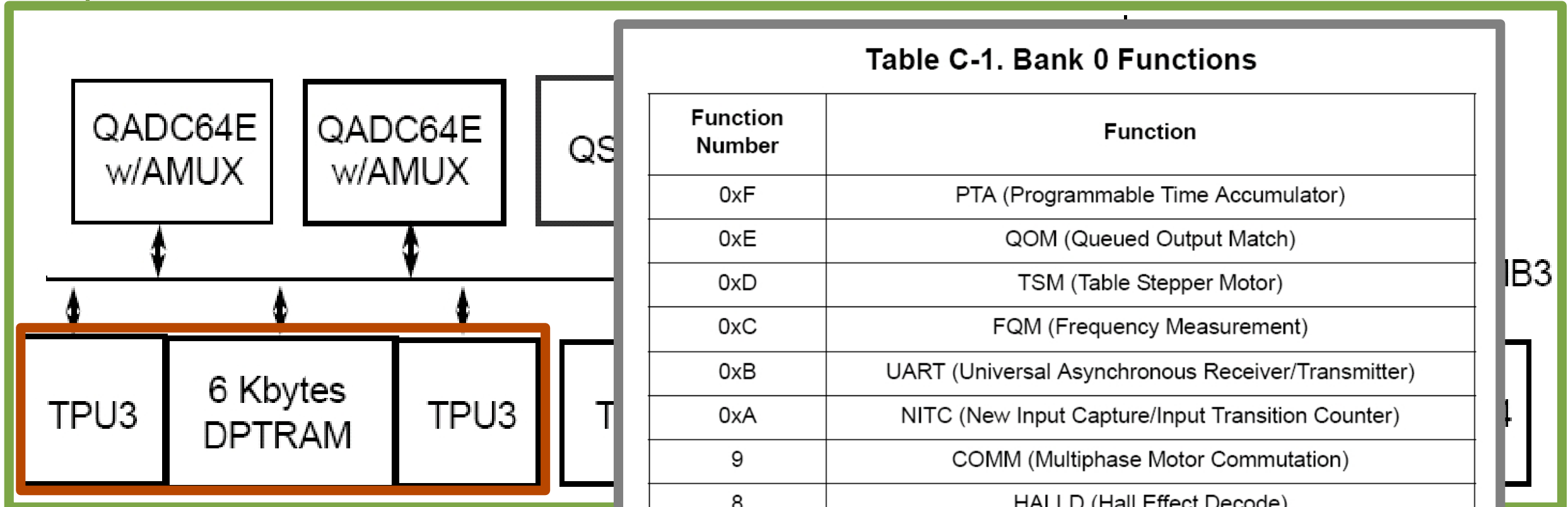
PWM

Input Capture

...

Zugriff auf Digital-Ein-/Ausgabe

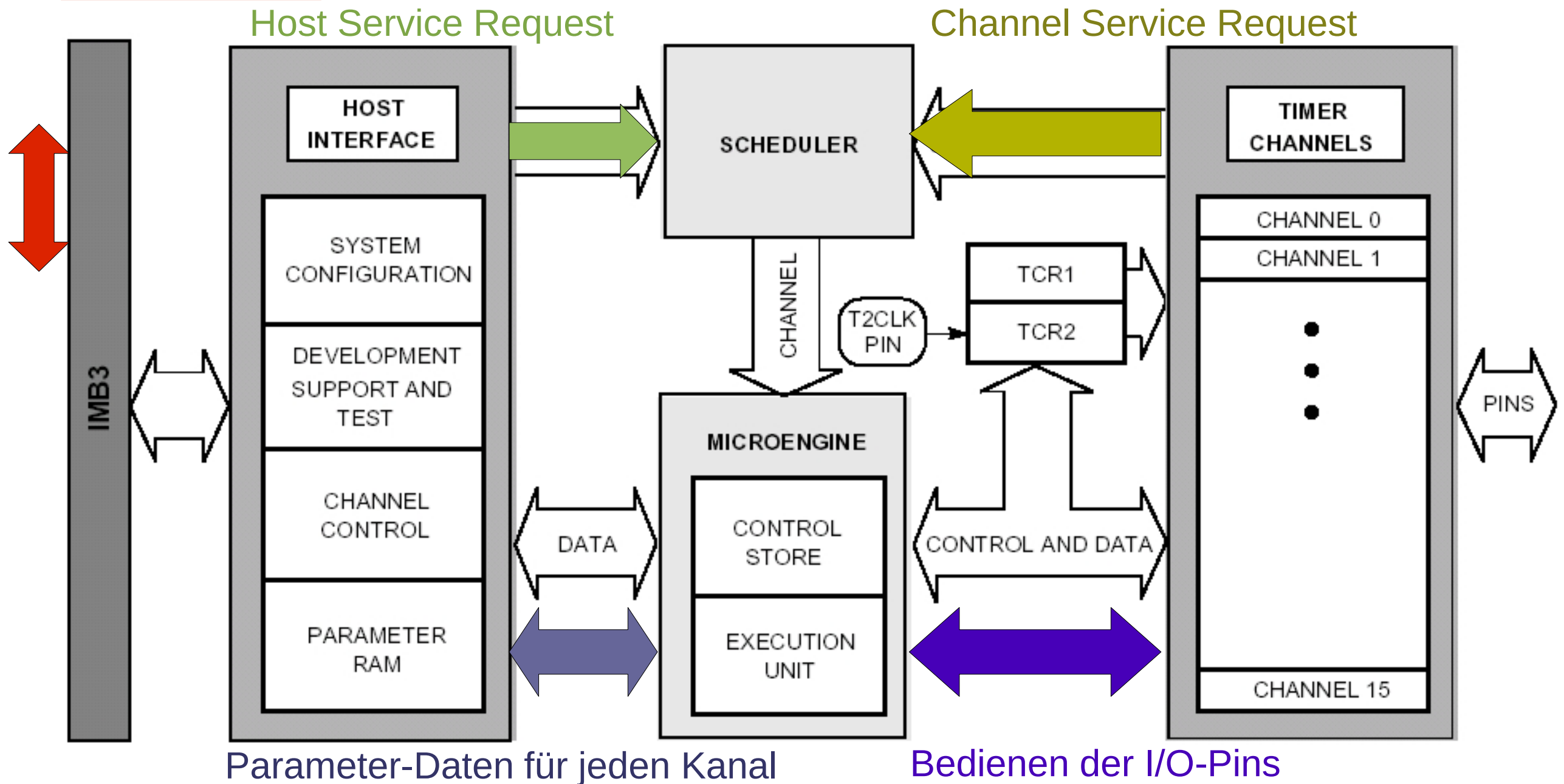
Peripherie



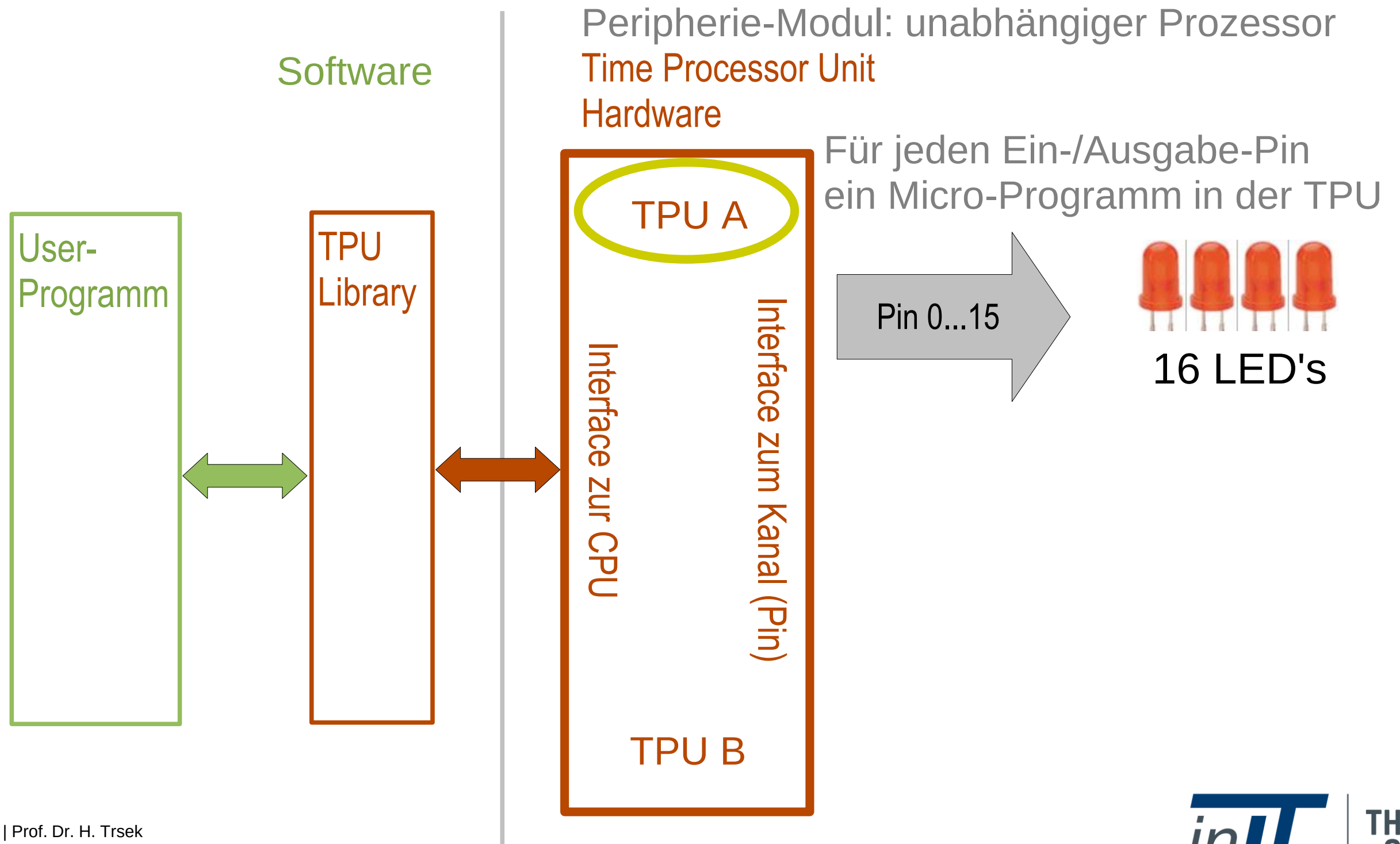
Time Processor Unit
Digital I/O
PWM
Input Capture
 ...

Zugriff auf Digital-Ein-/Ausgabe (TPU)

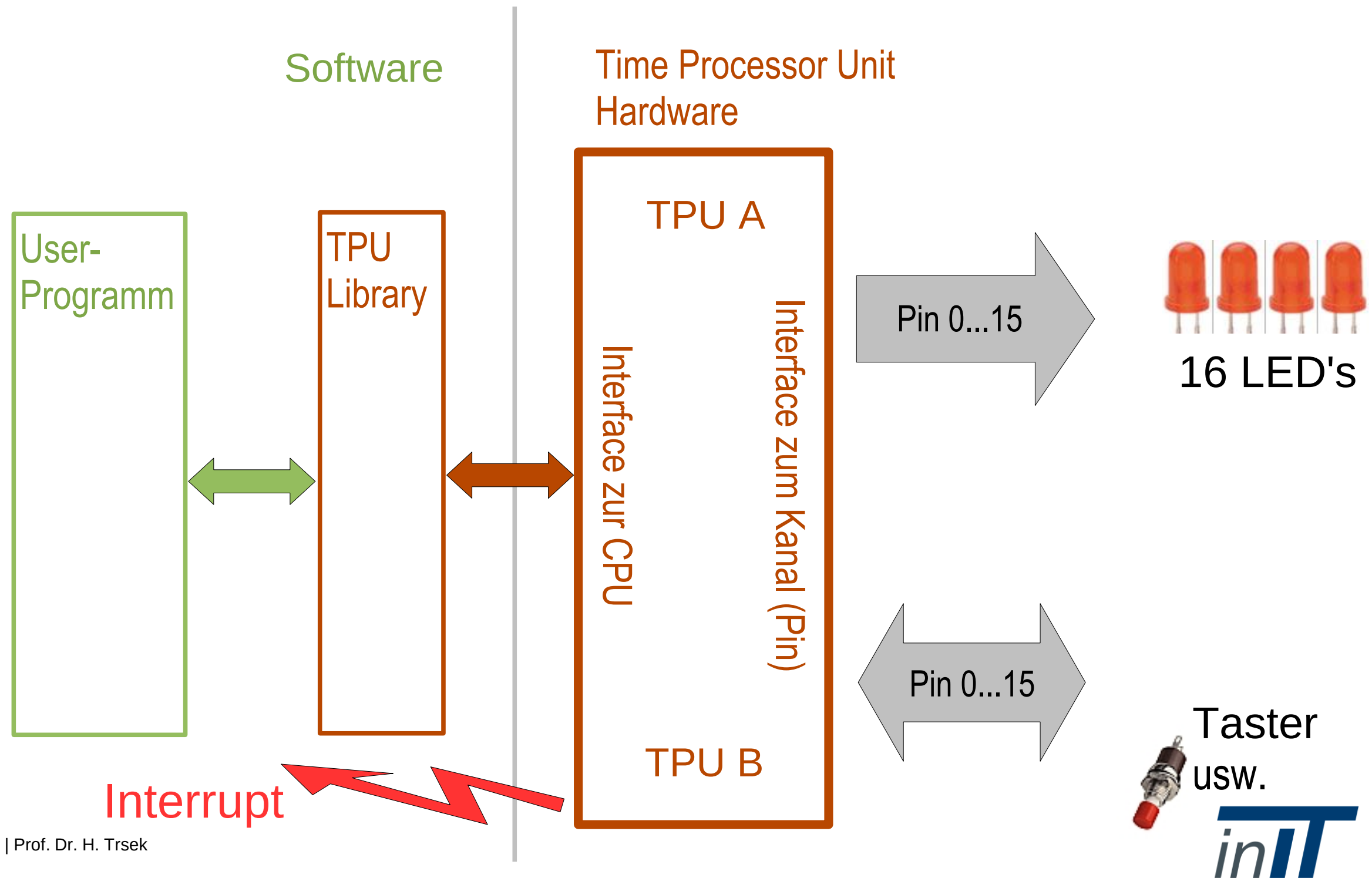
Prozessor-Kern
des MPC565



Zugriff auf Digital-Ein-/Ausgabe

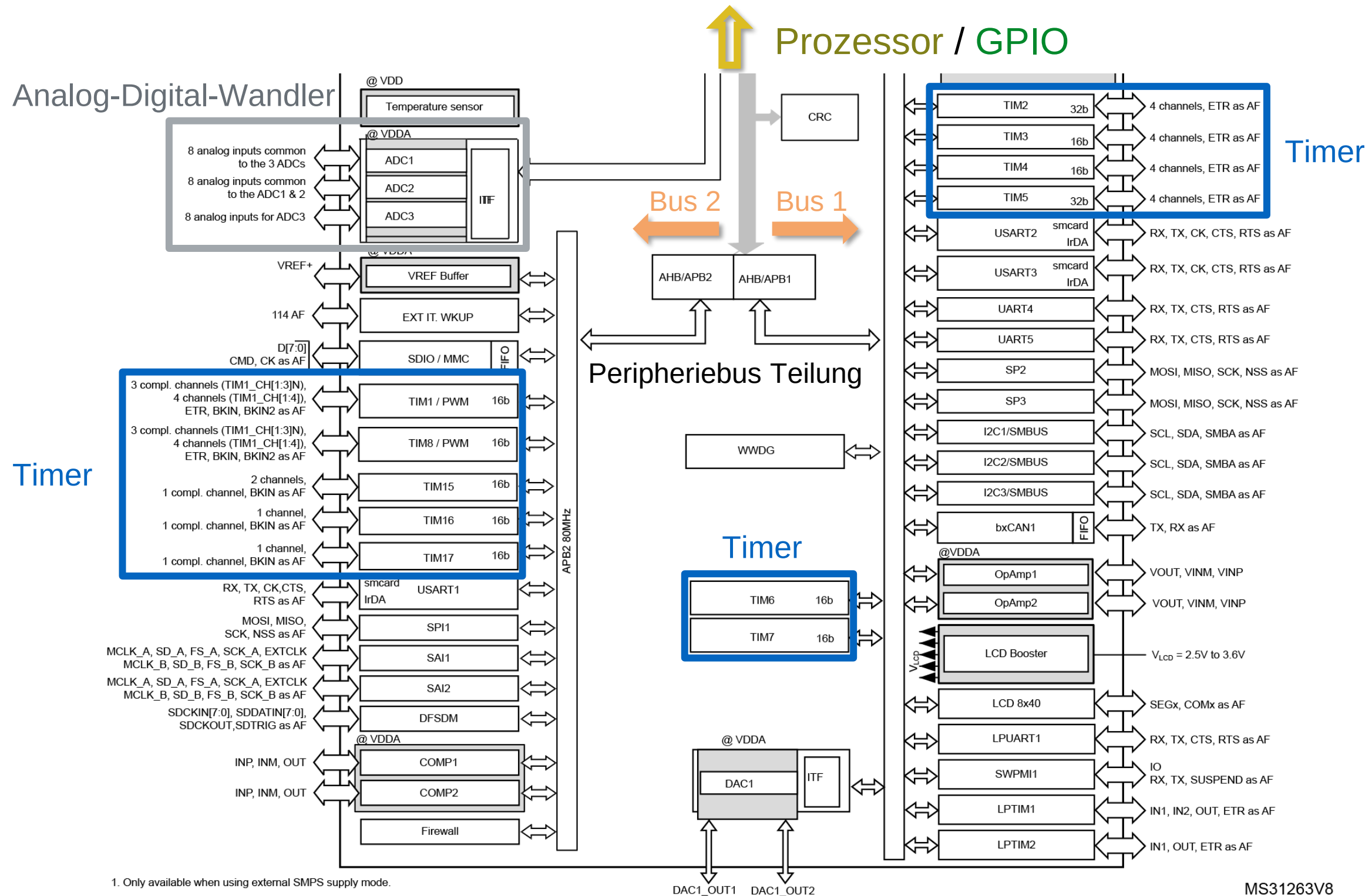


Zugriff auf Digital-Ein-/Ausgabe



Timer

Zugriff auf Digital-Ein-/Ausgabe



1. Only available when using external SMPS supply mode.

MS31263V8

Timer

0x5FFF FFFF
0x5006 0C00
0x4800 0000
0x4002 4400
0x4002 0000
0x4001 6400
0x4001 0000
0x4000 9800
0x4000 0000

Reserved
AHB2
Reserved
AHB1
Reserved
APB2
Reserved
APB1

Timers

APB2

...		
0x4001 4400 - 0x4001 47FF	1 KB	TIM16
0x4001 4000 - 0x4001 43FF	1 KB	TIM15
0x4001 3C00 - 0x4001 3FFF	1 KB	Reserved
0x4001 3800 - 0x4001 3BFF	1 KB	USART1
0x4001 3400 - 0x4001 37FF	1 KB	TIM8
0x4001 3000 - 0x4001 33FF	1 KB	SPI1
0x4001 2C00 - 0x4001 2FFF	1 KB	TIM1
0x4001 2800 - 0x4001 2BFF	1 KB	SDMMC1
0x4001 2000 - 0x4001 27FF	2 KB	Reserved
...		

Aufgaben von Timern

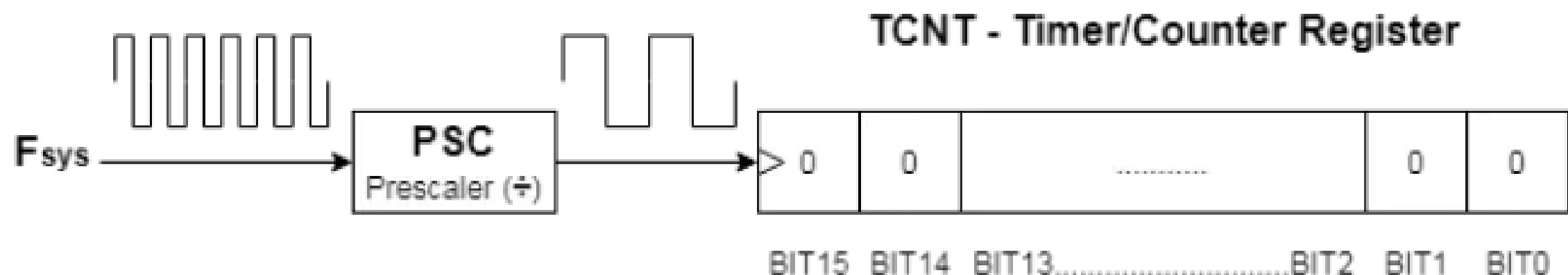
- ▶ Aufgaben von Timern
 - Bereitstellung von Zeitbasen
 - Erzeugung von Ereignissen
 - Erzeugung von Signalen
 - Zählen von ext. und int. Ereignissen
- ▶ Unabhängigkeit vom CPU Kern
- ▶ Unterschiedliche HW Timer im stm32
 - Basic timer
 - General purpose timer
 - Advanced timer

Optionen beim stm32

Timer	Typ	Bits	Prescaler (Bit)	Anzahl der Kanäle	Max. Inter- face Clock	Max. Timer Clock	APB
TIM1, TIM8	Advanced	16	16	4	SysClk/2	SysClk	2
TIM2, TIM5	GP	32	16	4	SysClk/4	SysClk, SysClk/2	1
TIM3, TIM4	GP	16	16	4	SysClk/4	SysClk, SysClk/2	1
TIM9	GP	16	16	2	SysClk/2	SysClk	2
TIM10, TIM11	GP	16	16	1	SysClk/2	SysClk	2
TIM12	GP	16	16	2	SysClk/4	SysClk, SysClk/2	1
TIM13, TIM14	GP	16	16	1	SysClk/4	SysClk, SysClk/2	1
TIM6, TIM7	Basic	16	16	0	SysClk/4	SysClk, SysClk/2	1

Basic Timer

- ▶ TIM6 und TIM7 sind 16-Bit-Timer
- ▶ Taktfrequenz kann durch einen Vorteiler (Prescaler) verringert werden.
- ▶ Vorteiler kann (ganzzahlige) Werte zwischen 1 und 65536 annehmen.
- ▶ Zählen nur aufwärts, also von 0 bis maximal 65535
- ▶ Max. Taktfrequenz durch Systemtakt vorgegeben

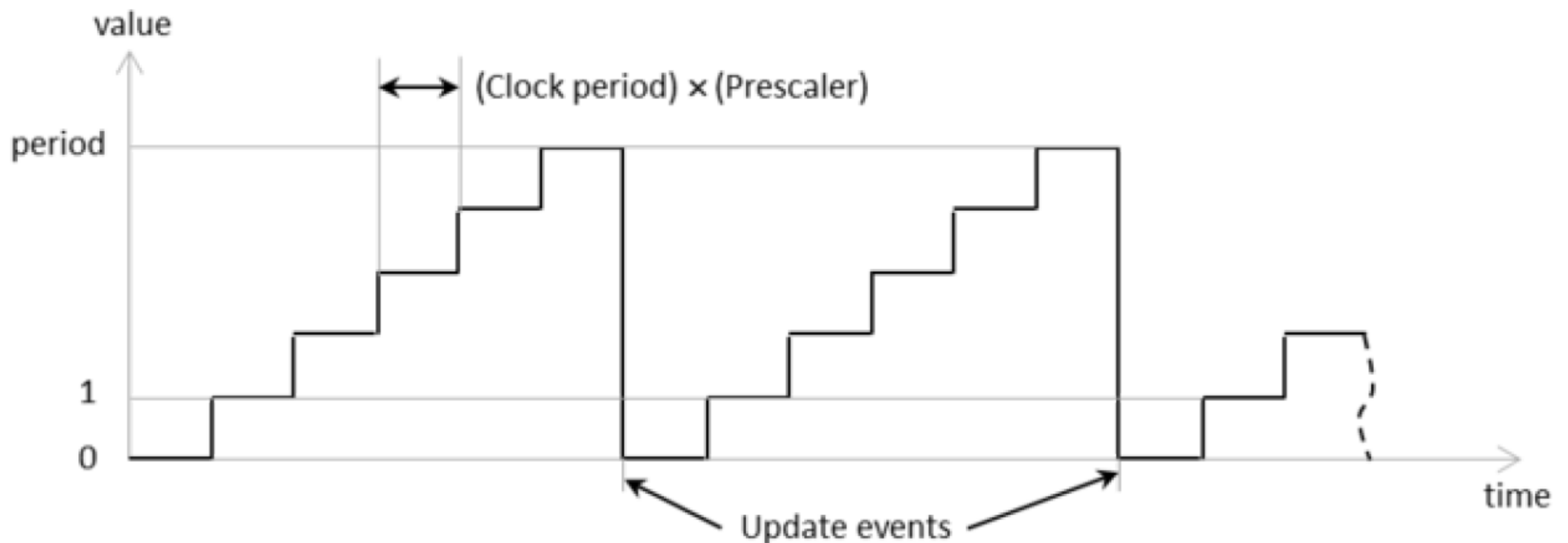


On-Chip-Peripherie – Timer Allgemein

Allgemeine Formeln

$$T_{tick} = \frac{1}{F_{Timer}}$$

$$T_{\text{überlauf}} = \text{ZählSchritte} * T_{tick}$$



Beispiel Basic Timer

```
// Aktivierung und Initialisierung von TIM6
RCC->APB1ENR |= RCC_APB1ENR_TIM6EN; // Bustakt aktivieren
TIM6->PSC = 1600-1;                    // Takt / 1600
TIM6->ARR = 10000-1;                   // Autoreload-Wert laden
TIM6->DIER |= TIM_DIER_UIE;           // Interrupt aktivieren
TIM6->CR1 |= TIM_CR1_CEN;              // Timer starten

NVIC_EnableIRQ(TIM6_DAC_IRQn); // TIM6: Interrupt aktivieren

TIM_GetCounter(TIM6); //Aktuellen Counter Wert holen

void TIM6_DAC_IRQHandler(void)
{
    tim6Trigger = true;
    TIM6->SR = 0;           // Wichtig: Muss per Software
                           // zurueckgesetzt werden!
}
```

Advanced Timer

- ▶ 16-bit up, down, up/down auto-reload counter
- ▶ 16-bit programmable Prescaler erlauben das Teilen der Clock Frequenz “on the fly” durch jeden Faktor zwischen 1 and 65536.
- ▶ Bis zu 4 unabhängige Kanäle für:
 - Input Capture
 - Output Compare
 - PWM generation (Edge und Center-aligned Mode)
 - One-pulse mode output

Advanced Timer

Reference Manual: Timers

TIM1/TIM8 introduction

The advanced-control timers (TIM1/TIM8) consist of a 16-bit auto-reload counter driven by a programmable prescaler.

It may be used for a variety of purposes, including measuring the pulse lengths of input signals (input capture) or generating output waveforms (output compare, PWM, complementary PWM with dead-time insertion).

Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers.

The advanced-control (TIM1/TIM8) and general-purpose (TIMy) timers are completely independent, and do not share any resources. They can be synchronized together as described in [Section 30.3.26: Timer synchronization](#).

...

PWM – Puls-Weiten-Modulation

PWM allgemein

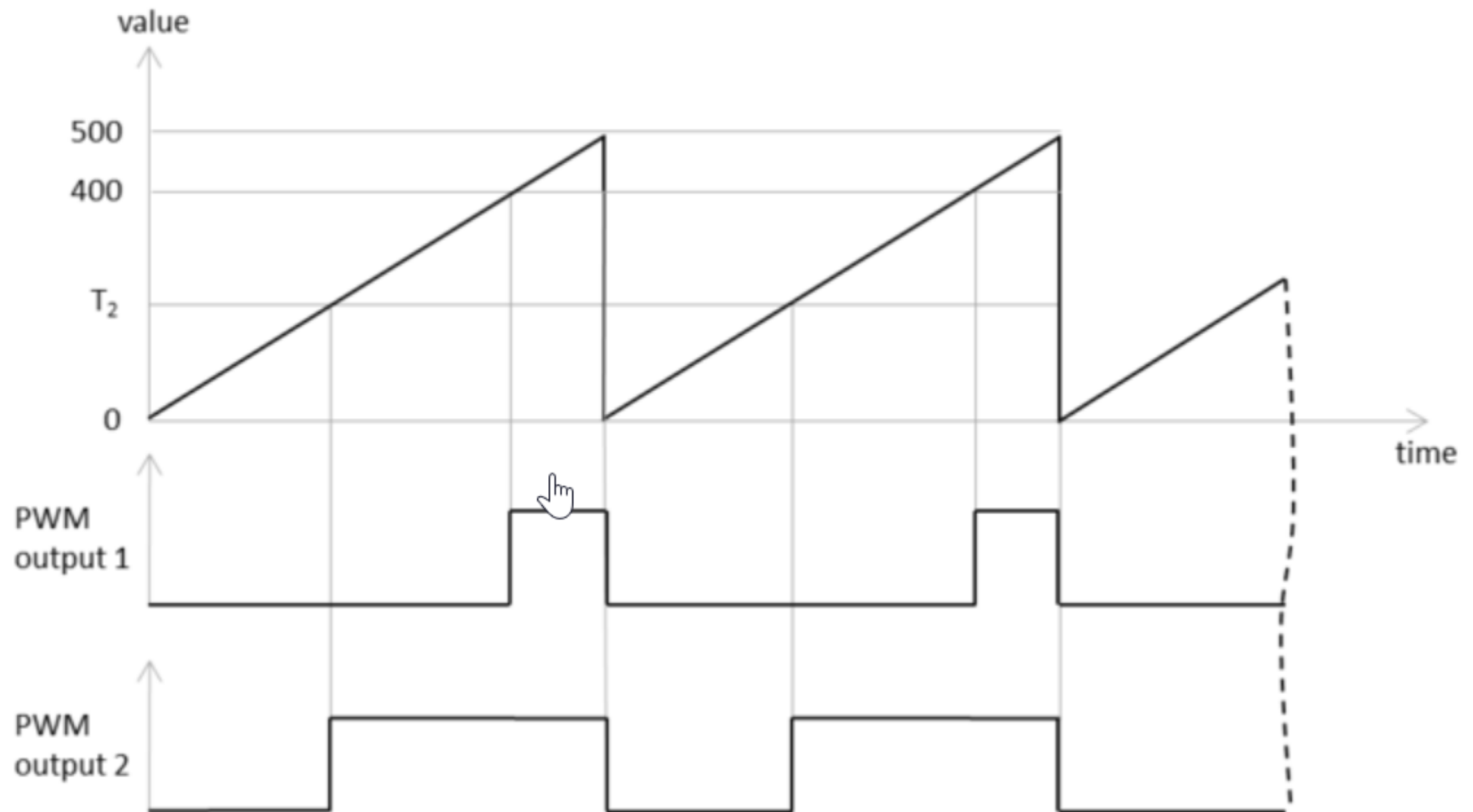
- ▶ Fürs messen von Zeiten nehmen wir immer Timer auf einem Mikrocontroller
- ▶ Aktivierung einer LED, fangen bei 0 an zu Zählen und warten
- ▶ Wenn der Timer einen definierten Wert erreicht hat (= gewisse Zeit verstrichen ist), schalten wir die LED wieder aus
- ▶ Warten bis Timer überläuft
- ▶ Wenn man das schnell genug macht, sieht man das als Mensch durch die Trägheit der Augen

PWM Beispiel

Beispiel

- ▶ Eine Timerperiode ist $50\mu\text{s}$ lang
- ▶ Annahme: der Timer zählt innerhalb dieser $50\mu\text{s}$ immer bis 500 und wird dann wieder auf 0 zurückgesetzt
- ▶ Setzen des OutputCompare auf 250
 - 50% der Zeit ist die LED aus und sonst an
 - $25\mu\text{s}$ LED an und $25\mu\text{s}$ LED aus
- ▶ Setzen des OutputCompare auf 100
 - 20% der Zeit ist die LED aus und sonst an
 - $10\mu\text{s}$ LED an und $40\mu\text{s}$ LED aus

PWM



PWM - Puls-Weiten-Modulation



- 2 PWM fähige Timer mit je 4 unabhängigen Kanälen
- programmierbare Periodendauer und Pulsweite
- PWM-Puls-Polarität für jeden Kanal
- wählbare Taktgeberquellen

PWM - Puls-Weiten-Modulation

- 2 PWM fähige Timer mit je 4 unabhängigen Kanälen
- programmierbare Periodendauer und Pulsweite
- PWM-Puls-Polarität für jeden Kanal
- wählbare Taktgeberquellen



PWM - Puls-Weiten-Modulation

- 2 PWM fähige Timer mit je 4 unabhängigen Kanälen
- programmierbare Periodendauer und Pulsweite
- PWM-Puls-Polarität für jeden Kanal
- wählbare Taktgeberquellen

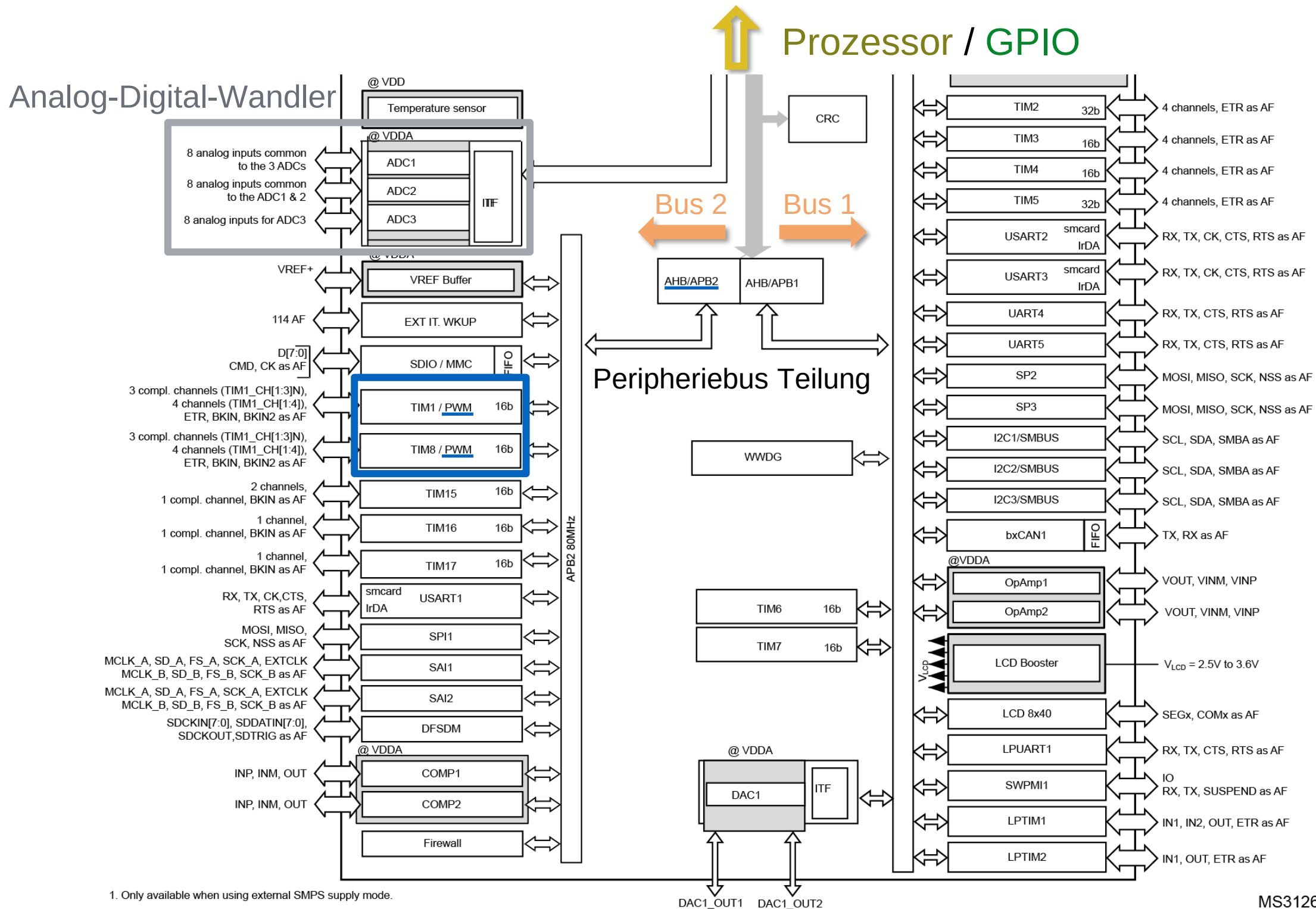


PWM - Puls-Weiten-Modulation

- 2 PWM fähige Timer mit je 4 unabhängigen Kanälen
- programmierbare Periodendauer und Pulsweite
- PWM-Puls-Polarität für jeden Kanal
- wählbare Taktgeberquellen

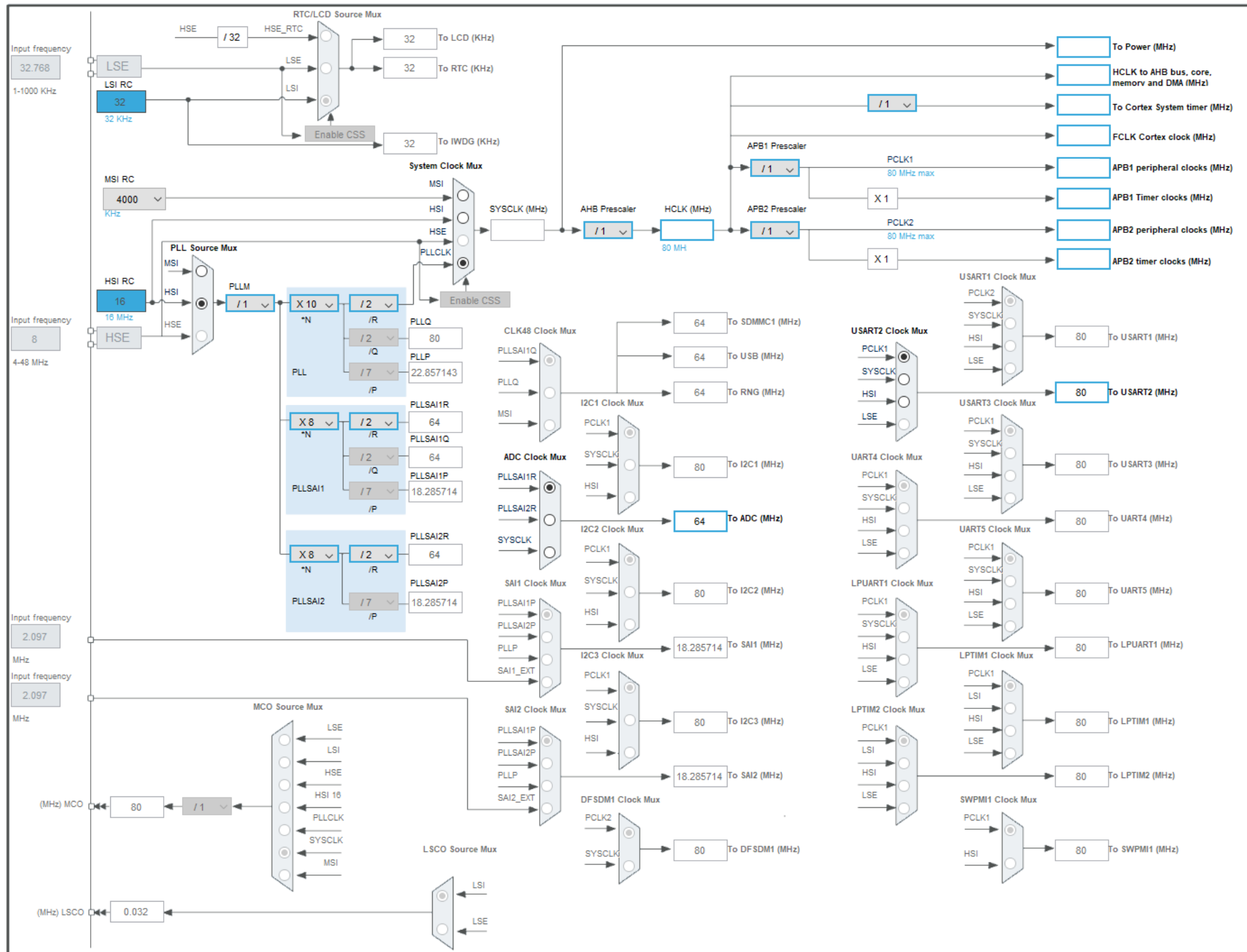


Zugriff auf Digital-Ein-/Ausgabe

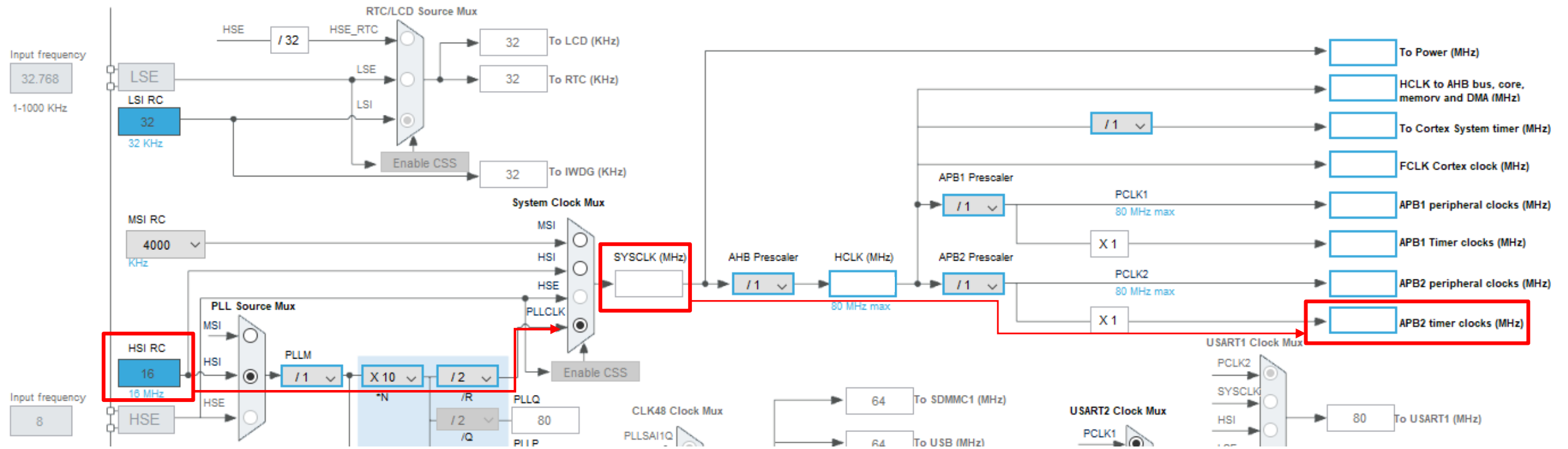


MS31263V8

PWM - Puls-Weiten-Modulation



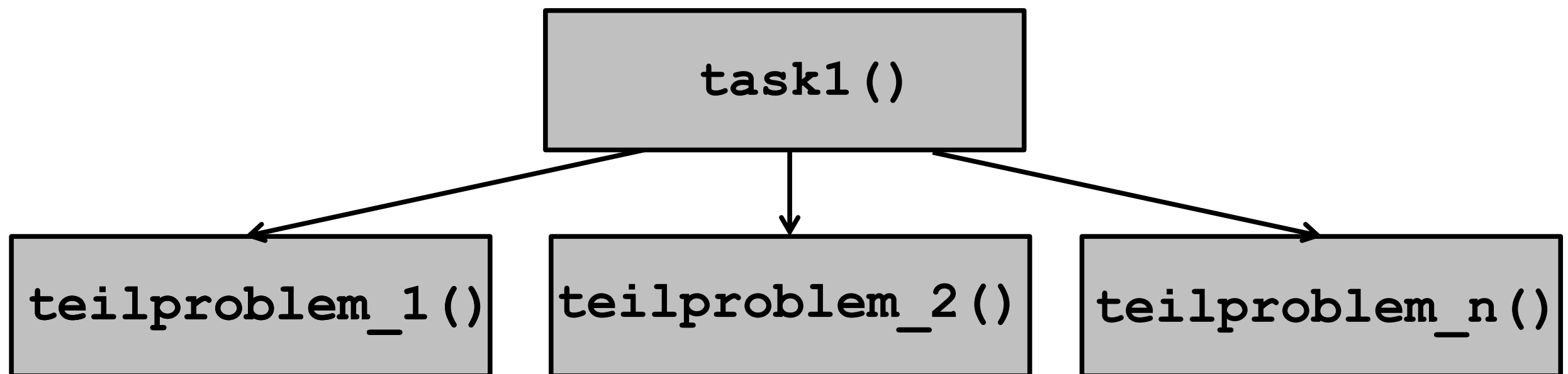
PWM - Puls-Weiten-Modulation



Funktionen

Funktionen (Unterprogramme)

- Strukturierung des Programms.
- Zusammenfassen von Teilen des Programms unter einem eigenen Namen.

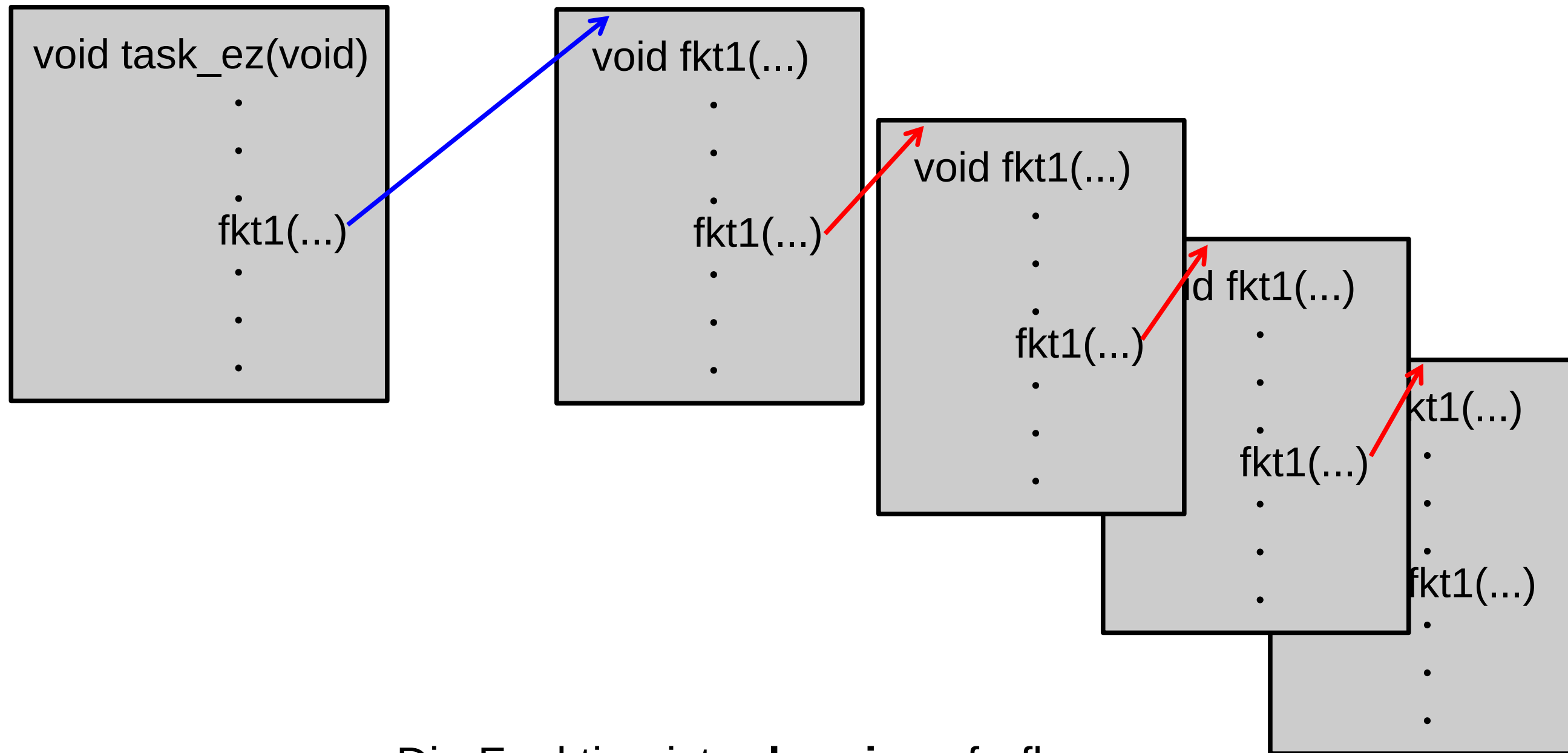


- jeder Sub-Algorithmus (Funktion) löst ein Teilproblem
- der Haupt-Algorithmus (Task) enthält die Ablaufsteuerung

Tasklaufzeit !

Funktionen (Unterprogramme)

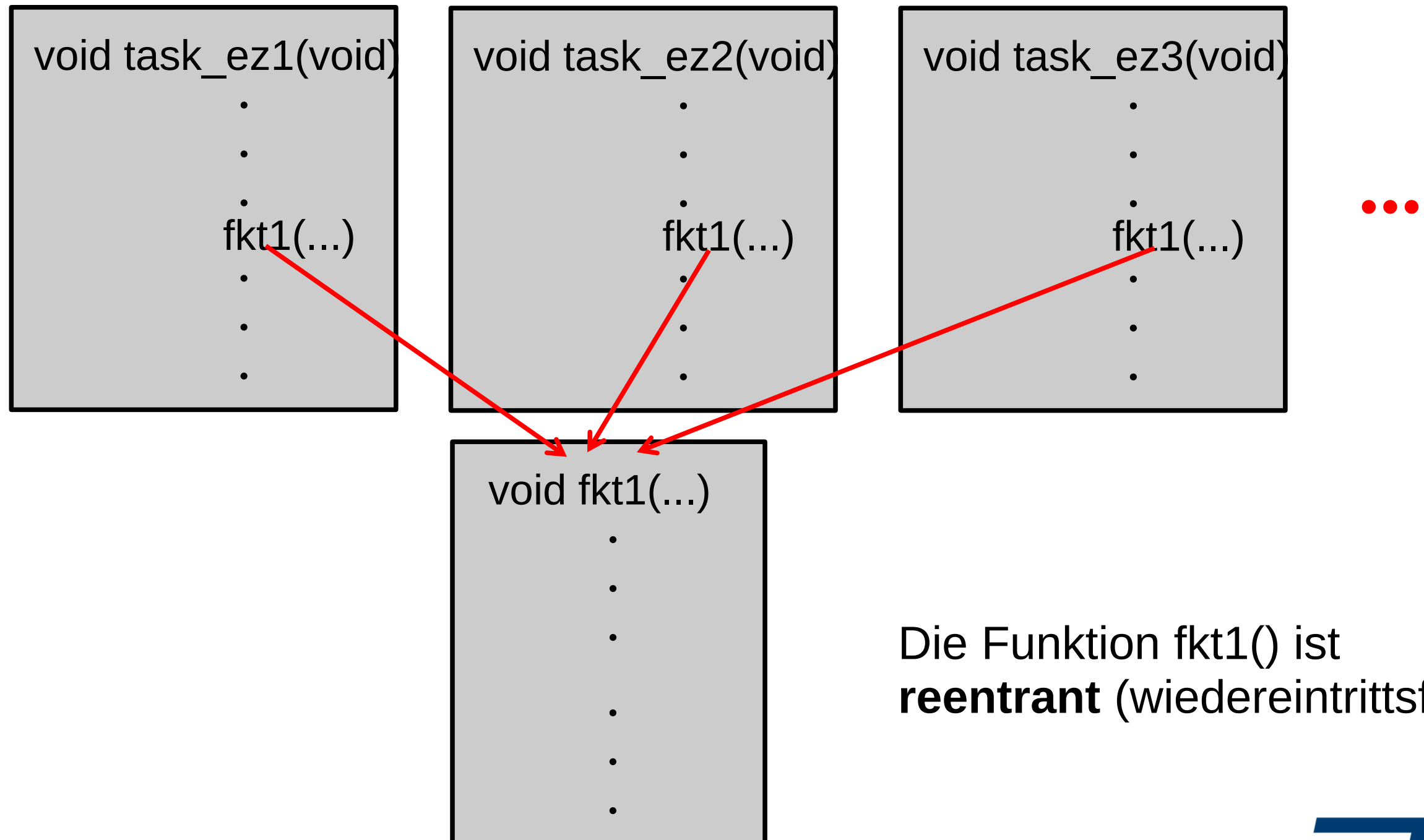
Funktionen im Multitasking



Die Funktion ist **rekursiv** aufrufbar.

Funktionen (Unterprogramme)

Funktionen im Multitasking



Die Funktion `fkt1()` ist **reentrant** (wiedereintrittsfest).